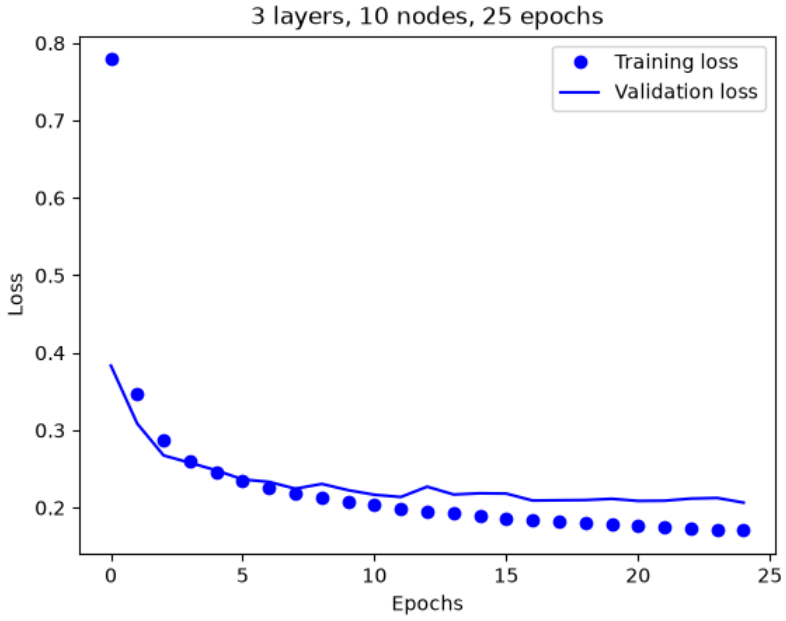
21 Assignment

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
import seaborn as sns
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

print(f"PyTorch Version: {torch.__version__}")
print(f"CUDA Available: {torch.cuda.is_available()}")

# Download the MNIST dataset
transform = transforms.ToTensor()
train_dataset = datasets.MNIST(root='./data', train=True,
download=True, transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False,
download=True, transform=transform)
train_loader = torch.utils.data.DataLoader(dataset=train_dataset,
batch_size=64, shuffle=True)
test_loader = torch.utils.data.DataLoader(dataset=test_dataset,
batch_size=64, shuffle=False)

class MyNetwork(nn.Module):
    #1a
    def __init__(self):
        super(MyNetwork, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=20)
        self.fc2 = nn.Linear(in_features=20, out_features=20)
        self.fc3 = nn.Linear(in_features=20, out_features=10)
        self.flatten = nn.Flatten()
        self.relu = nn.ReLU()
        self.softmax = nn.Softmax(dim=1)

    #1b
    def forward(self, x):
        x = self.flatten(x)
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x

#1c
model = MyNetwork()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

num_epochs = 25
loss_temp = []
loss_sum = 0
loss_values = []
val_loss_temp = []
val_loss_sum = 0
val_loss_values = []

for epoch in range(num_epochs):
    #1d
    model.train()
    for data, targets in train_loader:
        optimizer.zero_grad()
        outputs = model(data)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()
```

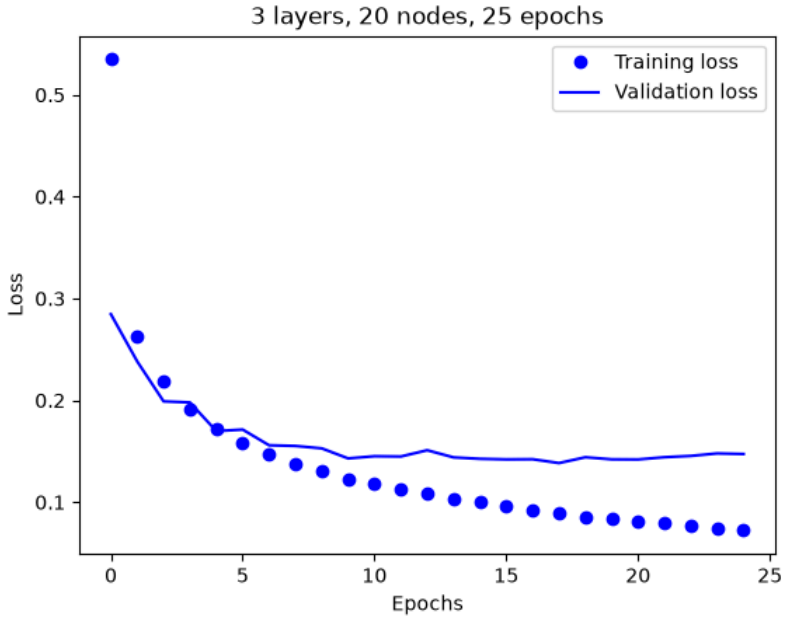
```
        loss_temp.append(loss.item())

    for i in range(0, len(loss_temp)):
        loss_sum += loss_temp[i]
    loss_values.append(loss_sum/len(loss_temp))
    loss_temp.clear()
    loss_sum = 0

    #1e
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for data, targets in test_loader:
            outputs = model(data)
            val_loss_temp.append(criterion(outputs, targets).item())
            _, predicted = torch.max(outputs.detach(), dim=1)
            total += targets.size(0)
            correct += (predicted == targets).sum().item()

        for i in range(0, len(val_loss_temp)):
            val_loss_sum += val_loss_temp[i]
        val_loss_values.append(val_loss_sum/len(val_loss_temp))
        val_loss_temp.clear()
        val_loss_sum = 0

    accuracy = 100 * correct / total

#1f
torch.save(model.state_dict(), 'model.pth')
print(f"Model saved with an accuracy of {accuracy}%")

plt.plot(list(range(num_epochs)), loss_values, 'bo', label='Training loss')
plt.plot(list(range(num_epochs)), val_loss_values, 'b', label='Validation loss')
plt.title('3 layers, 20 nodes, 25 epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.savefig('figure.png')
```

Results

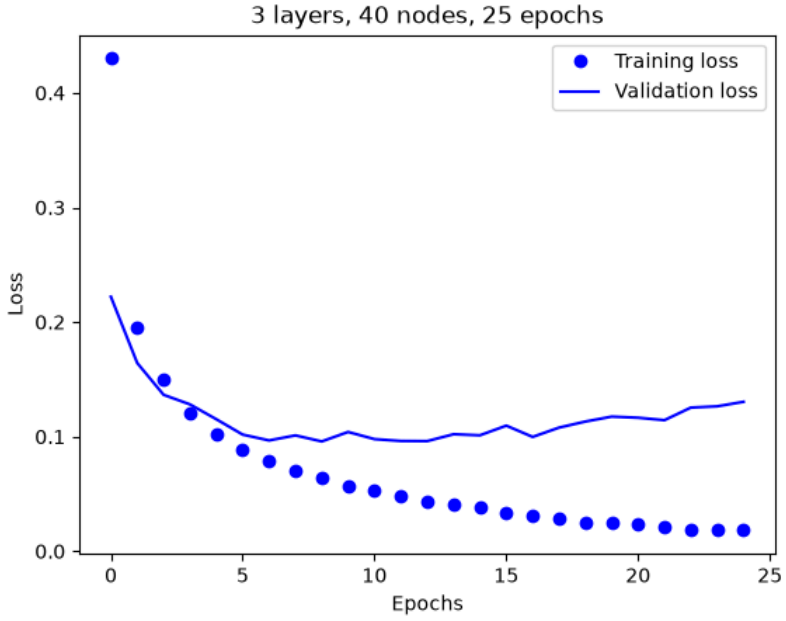
3 layers but changed node count



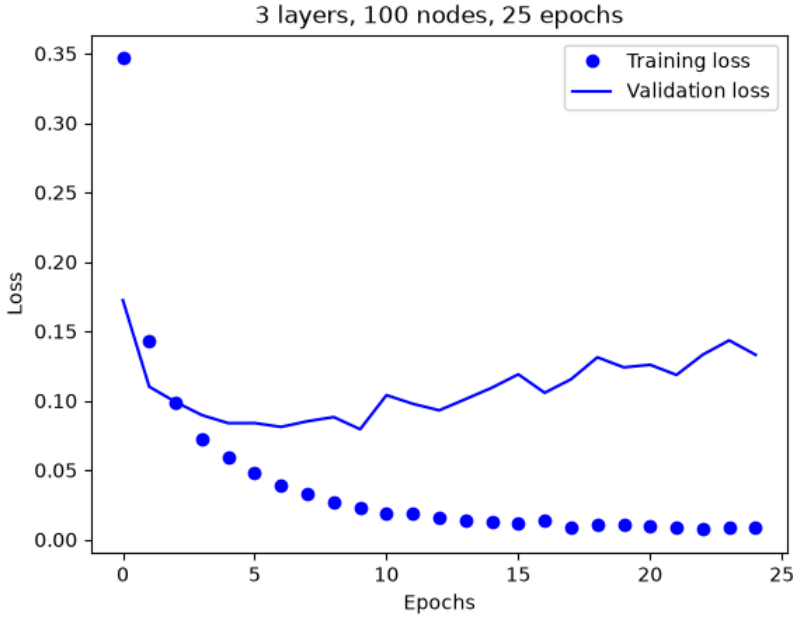
Resulting accuracy: 94.13%



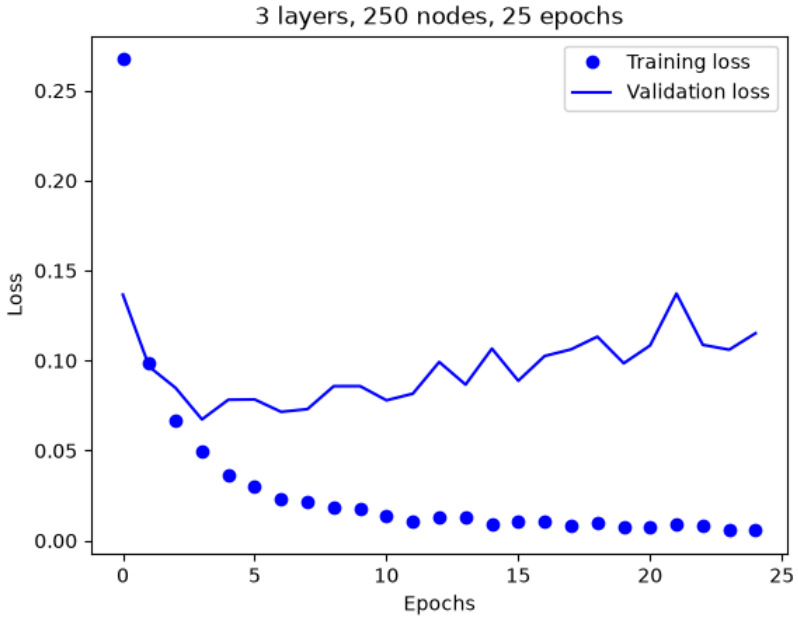
Resulting accuracy: 95.93%



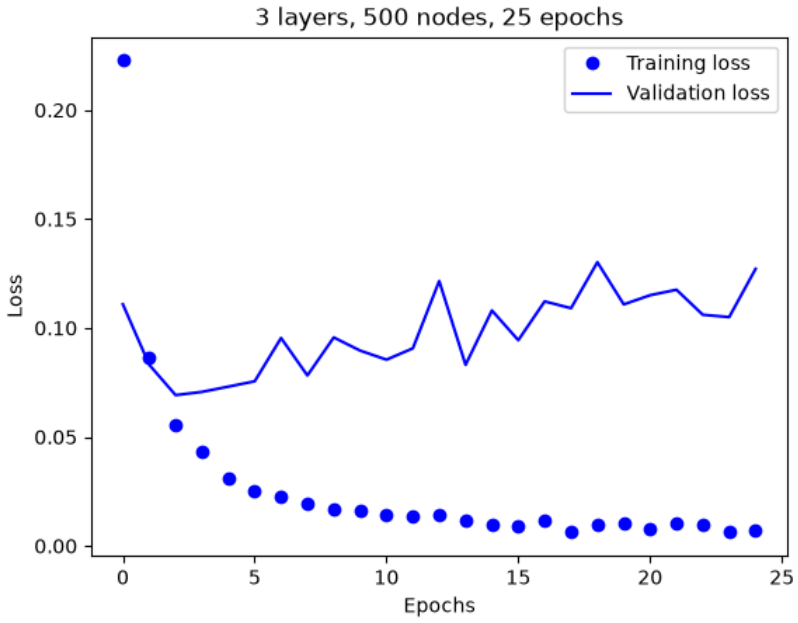
Resulting accuracy: 97.0%



Resulting accuracy: 97.72%

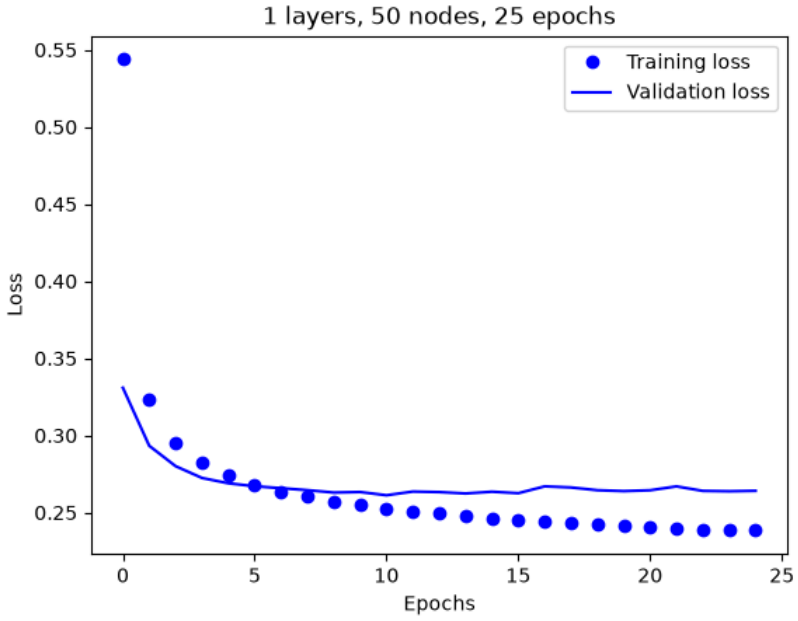


Resulting accuracy: 98.14%

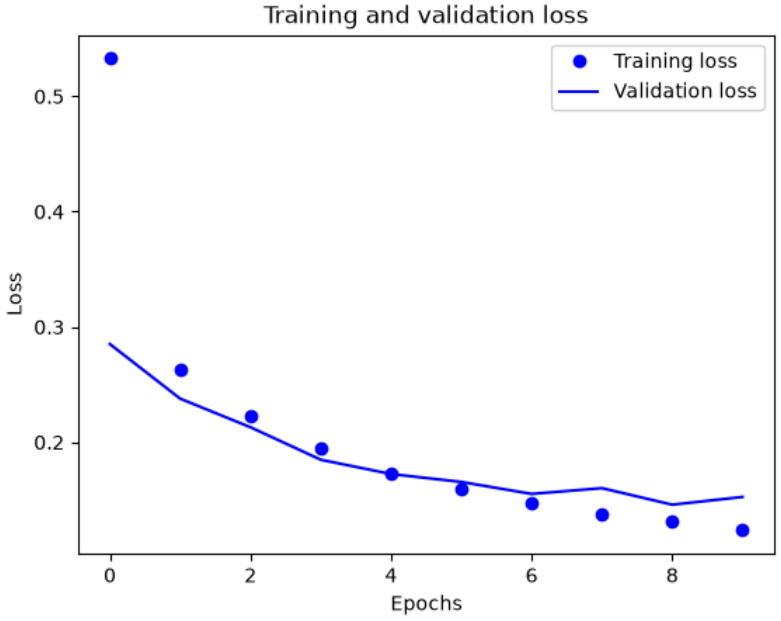


Resulting accuracy: 98.12%

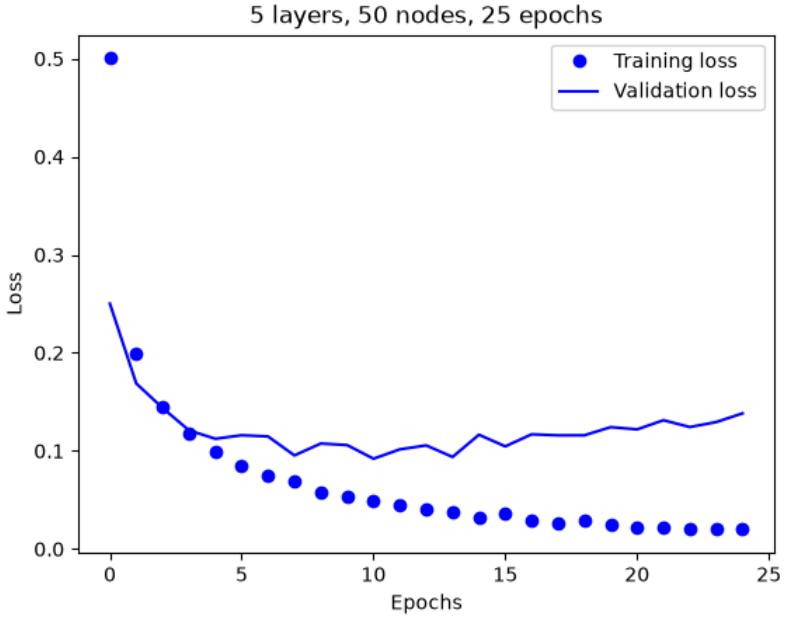
Layer changes



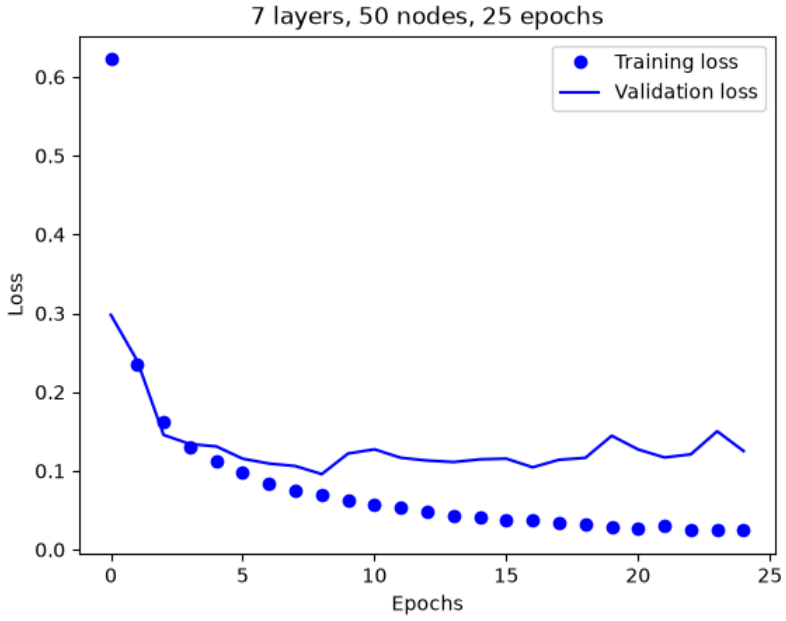
Resulting accuracy: 92.85



Resulting accuracy: 97.22%



Resulting accuracy: 97.05%



Resulting accuracy: 97.24%